Message Queues vs Event Streams

hand out work, or keep a log of what happened

Watch the 4-min video

In one line: a queue hands each message to one worker and deletes it when done, ideal for jobs; a stream is a durable, ordered log that many consumer groups read independently and can replay, ideal for events; many systems use both.

1 They are not interchangeable

RabbitMQ, SQS, Kafka, Kinesis: they all move messages from producers to consumers, so they seem interchangeable. They aren't. Some are **queues**, which hand out work and forget it once it's done. Others are **streams** (logs), which remember everything for a retention period and let anyone read from any point.

2 Queues: distribute work

Producers add messages; a pool of **competing consumers** takes them, so each message goes to one worker. The worker acknowledges after processing and the message is deleted; if it fails, the message becomes visible again and is retried, and after too many attempts it goes to a **dead-letter queue**. Queues track each message individually, which makes retries, delays and priorities easy. Scaling means adding workers.

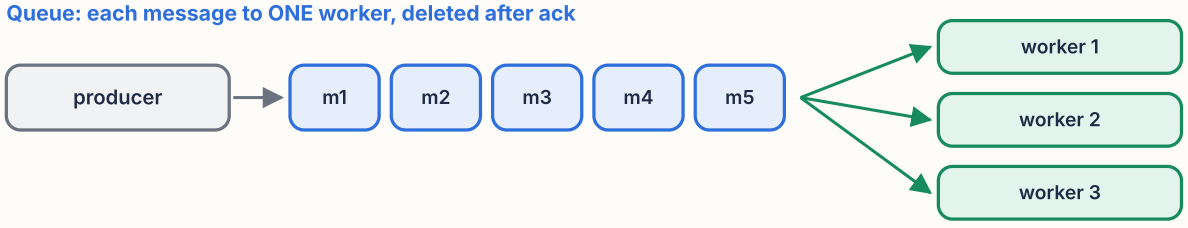


Figure 1. A queue load-balances messages across workers.

3 Streams: a shared log of events

A stream appends messages to an ordered, partitioned log that's kept for days or forever. Each **consumer group** tracks its own **offset** (position). Reading doesn't remove anything, so many independent groups read the same events, and a new service, or a fixed one, can **replay** history from the start. Ordering is guaranteed within a partition (lesson 26).

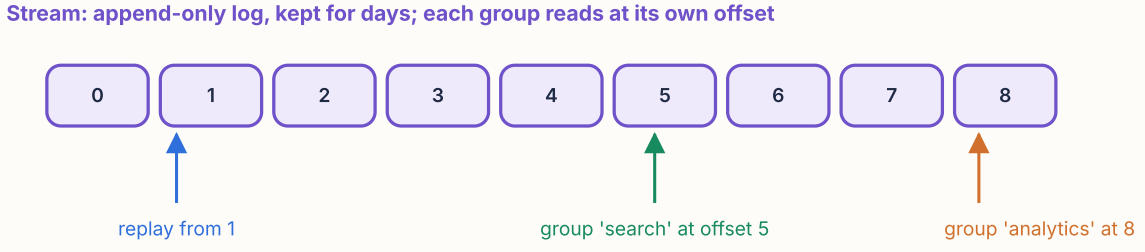


Figure 2. A stream keeps the log; each consumer group has its own position and can rewind.

	Queue	Stream
examples	RabbitMQ, SQS, Service Bus	Kafka, Kinesis, Pulsar, Event Hubs
after reading	message deleted	message kept (retention)
readers	competing workers share	many groups, each sees all
replay	no	yes, rewind the offset
ordering	best effort (FIFO options)	per partition
per-message	ack, retry, delay, DLQ	commit an offset

Figure 3. Queue vs stream at a glance.

WORKED EXAMPLE: USING BOTH

Email-sending jobs go to **SQS**: each job is picked up by one worker, retried a few times on failure, and moved to a dead-letter queue if it keeps failing; nobody needs it afterwards. Order events go to **Kafka**: the search indexer, analytics and the recommendations team each read every event in their own consumer group, and when analytics ships a bug fix they reset their offset to last Monday and reprocess a week of orders.

4 How to choose

- A **task** to be done once by someone (send email, resize image)? Queue.
- A **fact** that many services care about, or that you may want to replay or audit? Stream.
- Need per-message retries, delays and priorities? Queues make that easy; with streams you build it (lesson 32).
- Very high throughput and long retention? Streams.

COMMON MISTAKES

- Using Kafka as a job queue and fighting per-message retries and head-of-line blocking.
- Using a queue for events several teams need, then copying messages to many queues by hand.
- Expecting replay from a queue after messages were deleted.

5 Key terms

Queue: messages consumed once and deleted

Competing consumers: workers share one queue's messages

Offset: position in a stream partition

Stream / log: append-only, retained, replayable

Consumer group: readers that share one offset per partition

Retention: how long a stream keeps messages

6 Interview questions

Q1. What's the main difference between a queue and a stream? BASIC

A queue delivers each message to one consumer and deletes it after ack; a stream keeps an ordered log that many consumer groups read independently and can replay.

Q2. When would you pick RabbitMQ or SQS over Kafka? BASIC

For task distribution with per-message acks, retries, delays, priorities and dead-lettering, at moderate throughput, where nobody needs replay.

Q3. Order events must reach search, analytics and email. Queue or stream? SCENARIO

A stream: publish once, each service consumes in its own consumer group, and new consumers can be added or replay history without touching the producer.

Q4. A bug corrupted a week of analytics data. How does a stream help? SCENARIO

Fix the bug, reset the analytics consumer group's offsets to the start of the week, and reprocess the retained events.

Q5. Can Kafka act as a queue? TRICKY

Partly: a consumer group spreads partitions across consumers, but parallelism is capped by the partition count, one slow message blocks its partition, and per-message retry needs extra topics.

Q6. Why is ordering per partition, not per topic? TRICKY

Partitions let the log scale across brokers and consumers; a single total order would force everything through one sequence and one consumer.

Next: SD-I-003 · Producers and Consumers backendroadmap.com